

第 02 章 | 機器學習分類演算法：從感知器到梯度下降

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線與學習目標

本章節在機器學習的知識體系中具有關鍵的實務地位。本章會帶你回溯人工智慧的起點，見證演算法如何從最初模仿大腦神經細胞傳遞訊號的「生物靈感」，演進為現代機器學習核心的「數學最佳化」邏輯。從 1943 年的 MCP 神經元模型到 1950 年代的感知器，這段歷史不僅是技術的演進，更是將抽象的生物概念轉化為嚴謹運算模型的轉化過程。

為了銜接後續更複雜的分類器，本章的核心在於理解 誤差平方和 (Sum of Squared Errors, SSE) 如何作為橋樑，將我們從非黑即白的離散判斷，引導至可微、可最佳化的連續成本函數世界。

完成本章後，你可以達成以下四大核心學習目標：

理解 MCP 神經元與感知器的生物起源：了解早期科學家如何模仿樹突接收訊號、細胞本體整合以及軸突輸出的邏輯，並掌握其二元分類機制。

掌握感知器 (Perceptron) 與 Adaline 演算法的架構差異：辨析兩者在權重更新頻率（逐筆更新 vs. 批次更新）與激勵函數（離散步階 vs. 連續線性）的本質區別。

深入理解損失函數與梯度下降的數學直覺：建立「下山找谷底」的視覺化邏輯，掌握最小化 SSE 的數學原理。

學習使用 Python 與 NumPy 從零實作演算法：透過向量化運算 (Vectorization) 提升效能，並應用於鳶尾花 (Iris) 資料集進行分類驗證。

這些基礎演算法有助於理解深度學習；現代深層神經網路由大量神經元依層次連接而成。

2. 核心觀念：感知器 (Perceptron) 的直覺與限制

人工智慧的基石源於 1940-1950 年代對生物大腦處理電訊號與化學訊號的研究。

感知器的運作直覺：嚴格的守門員

我們可以將感知器想像成一個「評分機制」。當資料（特徵）進入模型時，各項特徵會被賦予權重。守門員計算「特徵 × 權重」的加總，得到一個「淨輸入」分數。

數學替代與簡化：為了計算方便，我們引入 偏誤項 (Bias Unit)。我們定義 $w_0 = -\theta$ (θ 為門檻值) 且對應的特徵 $x_0 = 1$ 。這使得原本的判斷式 $w_1x_1 + \dots + w_mx_m \geq \theta$ 可以簡化為： $z = w_0x_0 + w_1x_1 + \dots + w_mx_m = wTx$ 若 $z \geq 0$ ，預測為類別 1；反之為 -1。

重要術語中英對照表

中文術語	英文術語 (符號)	定義 / 作用
感知器	Perceptron	1957 年由 Rosenblatt 提出，能自動學習權重的二元分類演算法。
權重	Weights (w)	代表各個特徵對決策影響力的係數。
偏誤項	Bias Unit (w0)	基礎底分，數學上由 $w_0 = -\theta$ 換算而來，讓決策邊界能平移。
特徵	Features (x)	樣本屬性，如鸚尾花的花瓣長度。
淨輸入	Net Input (z)	特徵與權重的線性組合，即 wTx 。
激勵函數	Activation Function (ϕ)	決定神經元是否發射訊號；感知器使用「步階函數」。
學習率	Learning Rate (η)	更新權重時的步伐大小，通常介於 0.0 與 1.0 之間。
線性可分	Linearly Separable	資料點能被一條完美的直線（或超平面）完全切開。

感知器的收斂限制

感知器僅保證在資料為「線性可分」時收斂。若紅圈與藍叉資料互相混雜，演算法會為了一兩筆錯誤而不斷大幅修改權重，導致無限迴圈。因此，實作時必須設定「最大訓練回合數 (Epochs)」以確保程式能適時中止。

為了克服這種「僵化」且僅能處理二元分類的侷限，科學家隨後演化出更具彈性的 Adaline 演算法。

3. 演算法進階：Adaline 與梯度下降最佳化

Adaline (ADaptive LInear NEuron) 引入了「連續性激勵函數」，這是現代機器學習最佳化理論的起點。

感知器與 Adaline 的關鍵對比

更新依據：

Perceptron (增量更新)：根據「預測類別是否正確 (+1/-1)」來更新。每看一筆資料就更新一次。

Adaline (批次更新)：根據「連續預測分數與真實標籤的落差」來更新。它計算整個資料集的誤差後再統一最佳化。

成本函數：Adaline 旨在最小化誤差平方和 (SSE)： $J(w) = \frac{1}{2} \sum_i (y(i) - \phi(z(i)))^2$ 。

數學特性：Adaline 使用線性激勵函數 $\phi(z) = z$ ，使成本函數變為「連續且可微」的凸函數 (Convex)，讓我們能運用微積分求導。

梯度下降 (Gradient Descent) 的數學直覺

梯度下降可以比喻為「下山找谷底」。

原理：我們朝著梯度的「反方向」移動權重，以尋找 SSE 的全球最小值。

學習率 (η) 的兩難：這是步伐大小。步伐太大會發生「越過谷底」(Overshoot)，導致誤差不降反升，甚至數值溢位 (NaN)；步伐太小則效率過低，耗費運算資源。

4. 程式碼流程導讀：從物件導向打造分類器

在實作中，使用物件導向 (OOP) 建立類 Scikit-learn API 的類別，能幫助你養成標準開發習慣。

Perceptron 與 AdalineGD 類別導讀

初始化 (`__init__`)：

關鍵細節：權重 `w_` 不能設為 0。若全為 0，學習率 η 只能影響向量的「長度」而無法改變「方向」。因此我們必須使用極小的亂數（如常態分佈）來啟動。

學習核心 (`fit`)：

向量化操作 (Vectorization)：我們不使用緩慢的 `for` 迴圈逐一計算權重。

效能背後：程式碼中的 `X.T.dot(errors)` 利用了 NumPy 底層最佳化的 BLAS (基礎線性代數子程式) 與 LAPACK 函式庫，並支援 CPU 的 SIMD (單指令流多資料流) 指令集。這比純 Python 迴圈運算快上千百倍。

決策邏輯 (`predict`)：利用 `np.where` 實踐步階函數，將連續輸出轉回二元標籤。

5. 操作指南：鳶尾花資料集與特徵縮放

資料前處理 (Pre-processing) 往往比演算法本身更決定成敗。

課程實作清單與 Pro-tips

載入資料：使用 Pandas 載入 Iris 資料集。

學習提醒：本課程 Colab 直接載入公開鳶尾花資料，不需要準備本機 CSV。

資料視覺化：繪製散佈圖確認 Setosa 與 Versicolor 是否線性可分。

模型訓練：執行 `fit` 並觀察學習曲線。

特徵標準化 (Standardization) 的影響力

當特徵單位差異很大（如公分與毫米）時，SSE 成本函數的等高線會呈狹長橢圓形，使梯度下降路徑大幅震盪，降低收斂效率。

透過標準化（平均值 0、標準差 1），等高線會較接近圓形，權重更新路徑也會更直接。

延伸思考：多類別分類

雖然感知器與 Adaline 天生是二元分類器，但透過 One-vs.-All (OvA) 技術，我們可以為每個類別訓練一個分類器。預測時，選擇「淨輸入」絕對值最高的類別作為結果，即可解決多類別問題。

6. 常見錯誤與動手練習

常見錯誤與排除手冊

現象	可能原因	對策
Loss 變成 NaN 或無限大	學習率 (η) 過大，導致權重在山谷間彈跳發散。	將學習率調小一個數量級（如從 0.01 降至 0.001）。
學習曲線不降反升	未執行特徵標準化，或權重更新方向（正負號）錯誤。	檢查 X_{std} 標準化步驟；確認梯度更新是相加而非相減。
模型永遠無法收斂 (Error > 0)	資料為非線性可分，或迭代次數 (Epochs) 不足。	檢查散佈圖；對於非線性資料，感知器無法達到 0 誤差。

動手自我檢查清單

- ☐ 我能否解釋為何感知器是「增量更新」，而 Adaline 是「批次更新」？
- ☐ 我能否說明學習率過大如何導致 Overshoot 現象？
- ☐ 我是否理解 `X.T.dot(errors)` 內部如何利用 SIMD/BLAS 提升效能？
- ☐ 我能否推匯出偏誤項 w_0 是如何從門檻值 θ 轉換而來的？

本章總結：掌握感知器與梯度下降後，你已能說明模型如何更新權重。請嘗試不同的特徵組合（如花萼寬度與花瓣寬度），比較決策邊界的變化。下一章將使用 scikit-learn 的一致 API 套用這些原理。